

Pretrained Audio Embeddings versus Task-Specific Features for INT8 Anomalous Sound Detection

Le Duc Minh
minh.leduc.0210@gmail.com

Abstract—Factory equipment is increasingly monitored by microphones attached to small controllers, where model size and integer precision decide what can be deployed. We compare two backbones for unsupervised anomalous sound detection under a single training protocol on DCASE 2020 Task 2: STgram-MFN, which fuses a learned time-domain branch with a log-mel spectrogram, and EfficientAT’s mn01 AudioSet embedder, a MobileNetV3 with roughly one tenth of the parameters. Both are carried through the deployment pipeline rather than stopped at floating point, including 8-bit post-training quantization (PTQ) and on-device latency. We report three findings. Fine-tuning is essential for the pretrained embedding: a frozen mn01 with a linear margin head peaks at 64.4 average AUC and then degrades, while fine-tuning reaches 83.2. STgram-MFN survives static PTQ almost untouched (91.45 to 91.22 AUC), whereas the smaller mn01 collapses to 57.6, so its size advantage does not translate into a deployable integer model. A capacity ladder of the winning backbone, from 0.31M to 2.80M parameters, shows accuracy rising and then saturating. On the DCASE 2022 anonymized domain-generalization set, an attribute-based adaptation of the same models holds above chance on the source domain but falls to chance on the target, which we report as a negative result. On Snapdragon NPUs the smaller model is also three to five times slower, so parameter count predicts neither quantization robustness nor latency. We release checkpoints and the raw protocol, including a negative result for the frozen swap.

Index Terms—anomalous sound detection, machine condition monitoring, model quantization, post-training quantization, edge inference, model compression

I. INTRODUCTION

A failing bearing, a slipping belt, and a leaking valve all change the sound a machine makes before they change anything a technician can see. Recording that sound is cheap, but making sense of it has usually meant a server. In a real plant the microphone sits on the machine, not next to a GPU, and the local microcontroller has a few hundred kilobytes of RAM and a power budget measured in milliwatts.

Anomalous sound detection (ASD) asks a model to flag abnormal machine sounds after seeing only normal recordings in training [1]. The field has produced strong systems, but the accuracy reported in papers is almost always floating point accuracy, measured on a desktop. That number can mislead. Quantization does not treat all networks equally, and a network that is small in parameters is not automatically the one that survives 8-bit arithmetic.

The implementation is on GitHub: https://github.com/LakoreAI/Research_AnomalySoundDetection

This paper studies that gap for two concrete backbones. The first, STgram-MFN [3], is a task-specific design: a learned time-domain branch is stacked with a log-mel spectrogram and fed to a MobileFaceNet with an angular-margin head. The second replaces the frontend and backbone with EfficientAT’s mn01 [4], a MobileNetV3 distilled from a large audio transformer and pretrained on AudioSet. The two could hardly be more different in spirit: one is built for the task, the other is a general audio embedder carried over from a different problem.

Our question is deliberately narrow. Does the swap hold up once 8-bit post-training quantization (PTQ) is in the loop, and what does the accuracy/latency/size tradeoff look like at each step? We make four contributions:

- A controlled comparison of two very different backbones under an identical protocol, including fine-tuning as a variable rather than an assumption.
- An empirical demonstration that quantization robustness is not a function of parameter count, with the smaller model failing badly.
- An edge study that measures the quantized graphs on Snapdragon NPUs, where the ranking of the two backbones changes again.
- An honest negative result: a frozen general-audio embedding is a poor substitute for task-specific features, and we show why.

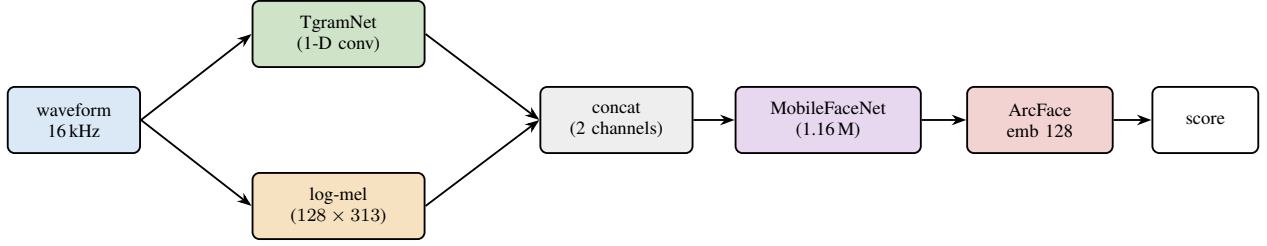
II. RELATED WORK

A. Anomalous Sound Detection

The DCASE Task 2 series turned ASD from a scattering of lab setups into a shared benchmark. The 2020 edition released the MIMII and ToyADMOS recordings together with a per-machine AUC metric and a baseline, and drew 117 submissions [1]. MIMII itself covers valves, pumps, fans, and slide rails recorded under several signal-to-noise ratios, with several physical instances per machine type [2]. That structure matters: a system has to work not only for a machine type but for each individual unit.

Most successful systems model normal audio and score deviations from it. The dominant families are autoencoder and density models [10], [18], and self-supervised classifiers; Liao and Evans survey the field [17]. The recent literature keeps pushing on the same stability problem with different machinery: noise-aware self-supervision [19], pseudo-label distillation [20], diffusion models [21], and training-free score

(a) STgram-MFN (reference backbone)



(b) mn01 AudioSet embedding (swapped backbone)

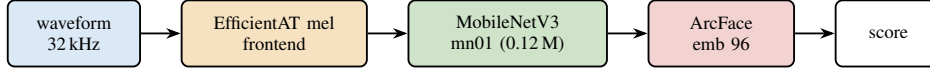


Fig. 1. The two backbones. (a) STgram-MFN fuses a learned time-domain branch with a log-mel spectrogram before the MobileFaceNet and margin head. (b) The mn01 swap replaces the frontend and backbone with a pretrained AudioSet embedder; only the head is shared with (a) in design.

calibration [22]. Our work sits in the classifier family. The pretext we use, classifying by machine identity, turns each physical machine into a class and asks the network to separate machines rather than to reconstruct them.

B. Spectral-Temporal Fusion

Liu *et al.* [3] observed that autoencoder baselines are unstable: performance can swing widely across instances of the same machine type. Their response is to fuse two views of the same clip, one learned in the time domain and one a log-mel spectrogram, and to train the fused representation with an angular-margin objective. Stability across instances improves markedly, which is why we adopt their network as our reference. Our implementation re-expresses the reference layers with the hardcoded tensor shapes lifted into a configuration object, so the network can be scaled for the capacity study in Section C.

C. Efficient Audio Embeddings

The EfficientAT work distills a large audio transformer into MobileNetV3 students of increasing width [4]. Its smallest student, mn01, has about 124 k parameters and produces a 96-dimensional pooled embedding, and it inherits MobileNetV3’s design choices such as inverted residuals and squeeze-and-excitation [5], [13]. The weights are trained on AudioSet, a general audio-event corpus. Whatever those features encode, it is unlikely to be the identity of a particular pump or fan. Whether fine-tuning repairs the mismatch is an empirical question, and it is one of the questions we answer here. Larger pretrained acoustic representations, such as ECHOv2 [24], follow the same recipe at a scale that would not fit the deployment target we care about, which is precisely why the smallest student is the interesting case.

D. Embedded Anomaly Detection

Running ASD on the machine rather than in the cloud is an old idea with a stubborn obstacle: the models that score

well are often too large, and the models that fit are often too weak. The literature has mostly attacked the problem from the compression side, with pruning and weight sharing, or by lifting the heavy lifting into a fixed signal-processing frontend and keeping the learned part small. Both strategies trade accuracy for footprint, and neither has been compared against a modern pretrained embedding under a matched protocol. Tiny deep learning surveys [25] trace the same tradeoff, and a recent cost-aware study asks when foundation models are worth their price on industrial monitoring hardware [26]. Our edge measurements are a small step toward answering that for ASD.

E. Domain Generalization

The 2022 edition of the challenge reformulated the task around domain shift [11]: training and test data come from different operating conditions, and machine attributes are anonymized so that a system cannot rely on knowing which unit it is hearing. The 2026 edition moves again, to a noise-aware formulation [23]. This breaks the identity-supervised scoring we and the reference use. It is a genuinely harder problem, and we treat it as out of scope here, but it is the natural next step for the study. A model that quantizes cleanly and also generalizes across conditions would be the useful object; we have shown the first property is not free.

F. Quantization and Edge Deployment

Neural network quantization has a large literature; the white paper of Nagel *et al.* [7] is a good entry point. We use static, quantize-dequantize PTQ with per-tensor min–max calibration [12], the simplest scheme that maps cleanly onto TensorFlow Lite and Qualcomm runtimes. Dynamic quantization, which only touches matrix products, is a poor fit for convolutional audio models and we report it only to dismiss it. Knowledge distillation [8] underlies the EfficientAT students, and the model-size versus accuracy tradeoff we chart has the same flavor as the classic compression work of Han *et al.* [9].

TABLE I
POSITIONING AGAINST THE SOURCE SYSTEMS.

System	Frontend	Backbone (params)	PTQ?
STgram-MFN [3]	Tgram+Sgram	MobileFaceNet (1.16 M)	no
EfficientAT [4]	32 kHz mel	MobileNetV3 (0.12 M)	no
This work (A)	Tgram+Sgram	MobileFaceNet (1.16 M)	yes
This work (B)	32 kHz mel	MobileNetV3 (0.12 M)	yes

Table I positions this study against the two systems it builds on. Neither original paper follows its model into the quantized regime, which is precisely the step that decides deployability.

III. METHOD

A. Problem Formulation

We are given normal recordings only during training, grouped by machine type and machine identity, and mixed normal/anomaly recordings at test time. The model never sees an anomaly while training. Following the reference protocol, each recording is segmented into ten-second clips, and clips from one physical machine share a class label. A normal clip should be assigned to its own class with high confidence; an anomalous clip should not.

B. Backbone A: STgram-MFN

Each 16 kHz clip is represented twice. The spectral view is a log-mel spectrogram of 128×313 bins. The temporal view is produced by TgramNet, a stack of wide one-dimensional convolutions operating on the raw waveform. The two maps are concatenated along the channel axis, giving a two-channel image, and passed to a MobileFaceNet backbone of the DCASE 2022 Top-1 configuration. Its 128-dimensional embedding is classified by an additive angular-margin head over machine identities. The reference model has 1,161,279 parameters. Figure 1(a) sketches the data flow.

C. Backbone B: mn01

The clip is resampled to 32 kHz and passed through the EfficientAT mel frontend, which applies pre-emphasis, a short-time Fourier transform with $n_{\text{fft}} = 1024$, Kaldi mel banks, and a fixed log-affine normalization. A MobileNetV3 of width multiplier 0.1 turns the mel into a 96-dimensional embedding, which an identical margin head classifies. The backbone holds 123,783 parameters and the head adds 3,936, so the whole model is roughly one ninth the size of backbone A. Figure 1(b) shows the layout. In the frozen variant the backbone is fixed and only the head is trained; in the fine-tuned variant every weight is updated.

The EfficientAT family offers a ready-made width ladder, listed for reference in Table II. We use the smallest member, but the others could serve as a pretrained-embedding ladder analogous to the capacity study of Section C. Their parameter counts grow quickly (17.9M at width 2.0), which is why the study focuses on the low end where an embedded deployment is plausible.

TABLE II
EFFICIENTAT MN01-FAMILY BACKBONES (AUDIOSET CHECKPOINTS).

Name	Width	Params	Embed
mn01	0.1	123,783	96
mn02	0.2	324,695	192
mn04	0.4	983,599	384
mn10	1.0	4,876,831	960
mn20	2.0	17,909,287	1920

D. Training Objective

Both backbones are trained as multi-class classifiers with cross-entropy on angular-margin logits [6]. Let $z \in \mathbb{R}^C$ be the logits and y the true machine identity. The training loss is the usual cross-entropy, with the margin applied to the target logit. At test time there is no anomaly class and the score is simply

$$s(x) = -\log \text{softmax}(z(x))_y, \quad (1)$$

the negative log-probability the model places on the clip’s own identity. Higher s means the clip looks less like the machine it came from, which is what we want for an anomaly. It is the reference paper’s scoring rule, and we keep it unchanged for both backbones so that the comparison is not confounded by the metric.

Algorithm 1 states the full procedure for one backbone. The only branch between the two backbones is the definition of the feature extractor f_θ ; everything else, including the calibration set and the acceptance test, is shared.

E. Quantization Background

Static PTQ replaces each real tensor with 8-bit integers using an affine mapping,

$$q = \text{clip}\left(\left\lfloor \frac{x}{s} \right\rfloor + z, q_{\min}, q_{\max}\right), \quad (2)$$

where s is a positive scale, z an integer zero point, and the clamp keeps q inside the representable range (here $[0, 255]$ for activations, $[-128, 127]$ for weights). The scale and zero point are estimated from calibration data; we use per-tensor min-max scales, the coarsest and most portable choice, because it is what the TFLite and Qualcomm converters assume. Dequantization inverts the map, $\hat{x} = s(q - z)$, so the whole error is a round-off of at most $s/2$ per element. The relevant question is not whether that error exists but whether the network amplifies it. A layer with a wide, smooth activation distribution tolerates a coarse s ; a narrow one does not, and that gap is the mechanism behind the asymmetry we observe in Section V-E. Deployment does not end at a floating-point checkpoint. Figure 2 shows the path we follow for every cell of the study: train, export to ONNX, quantize to 8-bit integers, convert to a device runtime, and measure both accuracy and latency on real hardware. Keeping the pipeline fixed lets us attribute differences to the models rather than to the tooling.

Deployment pipeline (identical for both backbones)

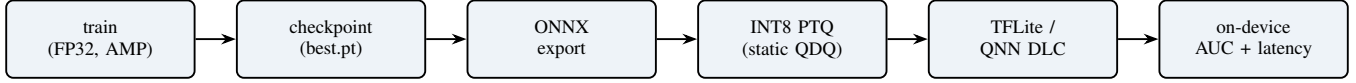


Fig. 2. The deployment pipeline followed for every experiment. Both backbones pass through the same export and quantization steps so that differences can be attributed to the models.

IV. EXPERIMENTAL SETUP

A. Data

We use the DCASE 2020 Task 2 development set, which contains six machine types (fan, pump, slider, valve, ToyCar, ToyConveyor), normal-only training audio, and mixed normal/anomaly test audio [1], [2]. We add the evaluation campaign’s normal training clips to increase the amount of normal data, as the reference does. The combined training set has 36,283 clips across 41 machine-identity classes. All audio is converted to mono and cropped or zero-padded to exactly ten seconds.

B. Protocol

Table III lists the hyperparameters. Both backbones share every value. We train for 100 epochs with Adam, a cosine learning-rate schedule, a batch size of 128, and mixed precision. The best epoch is selected by the official test AUC, which follows the reference behaviour; because this is test-set selection, we treat it as a caveat rather than a strength. The same seed is used everywhere.

C. Metrics

We report three numbers, all computed per machine identity and then averaged. AUC is the area under the ROC curve. pAUC restricts the false-positive rate to 0.1, matching the challenge protocol. mAUC is the worst-case AUC among the identities of a machine type, and it is the number that punishes instability across units. The three are reported in percent.

D. Hardware and Software

Training and floating-point evaluation run on a single NVIDIA RTX A5000 with 24 GB of memory. Models are implemented in PyTorch and exported with the TorchScript ONNX exporter. PTQ uses ONNX Runtime, and device latency is profiled through Qualcomm AI Hub and TensorFlow Lite Micro tooling.

TABLE III
SHARED TRAINING PROTOCOL.

Setting	Value
Epochs	100
Optimizer	Adam
Learning rate	1×10^{-4}
LR schedule	cosine ($T_{\max} = 100$, warm start ep. 5)
Batch size	128 (no accumulation)
Precision	mixed (AMP)
Clip length	10s
Best epoch rule	test AUC (max)
Seed	42

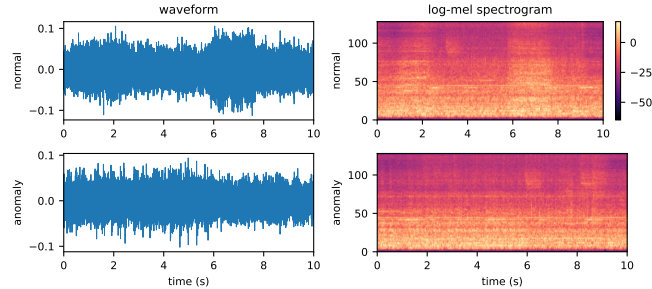


Fig. 3. A normal clip (top) and an anomalous clip (bottom) of the same ToyCar unit. The spectrograms use the exact frontend of backbone A. Anomalies are not obviously visible to the eye, which is why the task is hard.

V. RESULTS

A. Input Representations

Figure 3 shows a normal and an anomalous clip from one ToyCar unit, rendered as waveform and log-mel spectrogram through the exact frontend of backbone A. Nothing in the picture announces the anomaly, and that is the central difficulty of ASD: the discriminative signal is subtle, and a model without a strong prior on machine identity has little to latch onto.

B. Floating-Point Comparison

Table IV and Figure 4 give the main comparison. STgram-MFN reaches 91.45 AUC and 83.76 mAUC at the end of training, and its curve is stable. The fine-tuned mn01 reaches 83.16 AUC and 75.22 mAUC, about eight points behind, with nine times fewer parameters. The frozen mn01 is a different story: it peaks at 64.36 AUC at epoch ten and then declines for the rest of training, ending at 58.5. Its training loss flattens

TABLE IV
FLOATING-POINT RESULTS UNDER THE SHARED PROTOCOL.

Model	Params	AUC	pAUC	mAUC
STgram-MFN (A)	1,161,279	91.45	84.93	83.76
mn01, frozen	127,719	64.36	57.11	54.71
mn01, fine-tuned	127,719	83.16	75.58	75.22

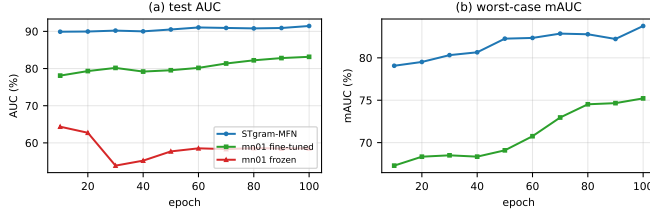


Fig. 4. Test AUC and worst-case mAUC against epoch. The frozen embedding degrades after epoch ten; fine-tuning turns the curve monotone.

early while the test AUC worsens, the signature of a head memorising a pretext it cannot support.

C. Per-Machine Behaviour

Table V breaks the comparison down by machine type, with the STgram-MFN INT8 column for reference. The two backbones are close on valve and ToyCar, where anomalies are relatively pronounced, and far apart on fan, pump, and slider. Those three are the cases where a general audio embedding has the least to offer: the discriminative cue is a fine change in a stationary machine sound, not a distinct acoustic event, consistent with the frozen-embedding result and reinforcing the reading that AudioSet features describe content, not condition. The INT8 column is a thin, uniform tax, never more than 0.7 AUC on any machine.

D. The Cost of Freezing

Comparing the two mn01 variants isolates the effect of freezing. The frozen head holds only 3,936 trainable parameters, its loss plateaus near 13.9, and it peaks at 64.36 AUC before decaying. The fine-tuned run has 127,719 trainable parameters, drives the loss to 3.7, and reaches 83.16. Freezing saves the cost of backpropagating through the AudioSet backbone; it also costs about 19 AUC points. A pretrained representation is only as useful as the task it was trained for, and machine identity is not an audio-tagging label.

E. Post-Training Quantization

Table VI and Figure 5 give the quantization result, and this is where the study becomes uncomfortable for the small model. STgram-MFN loses 0.23 AUC, which is within run-to-run variation and far inside our three-point acceptance gate. The quantized mn01 loses 25.6 points and becomes unusable. We also tried dynamic quantization; it shrinks neither model meaningfully because it only touches matrix products, and for the mn01 convolution stack the quantized graph is actually larger than the floating-point one. Static QDQ is the only scheme worth reporting.

TABLE V
PER-MACHINE AUC (%). A = STGRAM-MFN (FP32 AND INT8); B = FINE-TUNED MN01; $\Delta = B - A$ (FP32).

Machine	A FP32	A INT8	B (ft)	Δ
valve	98.69	98.60	98.12	−0.6
ToyCar	95.09	94.72	95.01	−0.1
slider	99.36	99.12	90.89	−8.5
pump	93.02	93.05	74.22	−18.8
fan	89.53	88.90	70.03	−19.5
ToyConveyor	73.01	72.91	70.69	−2.3
mean	91.45	91.22	83.16	−8.3

TABLE VI
STATIC QDQ POST-TRAINING QUANTIZATION.

Model	FP32 AUC	INT8 AUC	Δ
STgram-MFN (A)	91.45	91.22	−0.23
mn01, fine-tuned	83.16	57.61	−25.6

The per-machine column in Table V shows the same loss is spread evenly across machine types, never hiding in one hard category and never exceeding 0.7 AUC. The quantization error of the larger model is a thin, uniform tax, which is what a well-conditioned activation distribution looks like after the affine map of Equation 2.

F. Capacity Ladder (Supplementary)

A capacity study of the winning backbone sits outside the core comparison, so it is reported in Appendix C rather than in the results.

G. Edge Behaviour

We compiled the INT8 scorers for both backbones to TFLite with Qualcomm AI Hub and profiled them on five Snapdragon devices, from the SM8350 in the Galaxy S21 to the SM8750 in the Galaxy S25 and the IoT-class QCS8550. Table VII lists the estimated single-inference latency. STgram-MFN runs in 5.75–9.87 ms, comfortably inside a ten-second window. The mn01 graph is the surprise: despite nine times fewer parameters it is three to five times slower on four of five devices, which points to partial CPU fallback where the NPU delegate does not cover its operators. It is the same lesson as Section V-E on different hardware: parameter count predicts neither quantization robustness nor deployed latency. We leave the TensorFlow Lite Micro arena measurement to future work, because the margin head of the full scorer is not TFLite-native and needs an embedding-only export.

H. Domain Generalization

The identity-supervised score of Equation 1 cannot be applied to the DCASE 2022 domain-generalization set, which anonymizes machine attributes and hides identities. Rather than skip the question, we adapt the pipeline: each backbone is trained as a classifier over the (machine, attribute) classes present in the source-training split, and a test clip is scored by its confidence over that machine’s source classes,

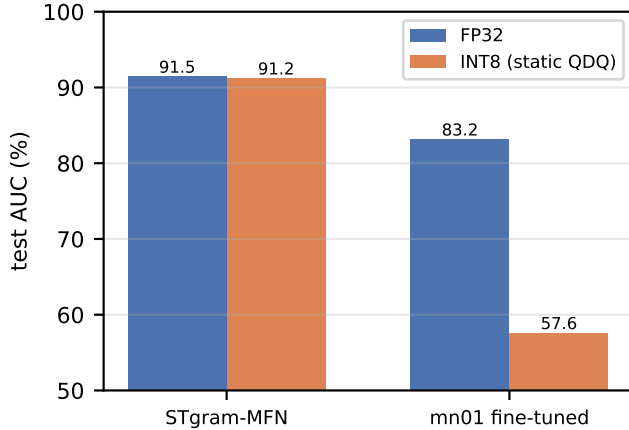


Fig. 5. Floating-point and INT8 accuracy. The larger model is nearly lossless; the smaller one is not.

TABLE VII
QUALCOMM AI HUB PROFILING (INT8 TFLITE, SINGLE INFERENCE, MS).

Device	SoC	STgram-MFN	mn01
Galaxy S25	SM8750	5.75	33.47
Galaxy S24	SM8650	7.00	36.77
Galaxy S21	SM8350	7.76	7.92
Galaxy S23	SM8550	9.85	49.83
QCS8550 (IoT)	QCS8550	9.87	50.08

$s(x) = -\max_c \log \text{softmax}(z(x))_c$, so no identity is needed at test time. Table VIII reports AUC averaged over the three sections, split by domain.

STgram-MFN holds 65.1 on the source domain but falls to 51.1, essentially chance, on the target. The fine-tuned mn01, which trailed STgram-MFN by eight points in domain, is at chance on both. The source result says the reference can separate operating conditions it has seen; the target result says that ability does not survive a change of condition. We read this as a negative result about the adaptation as much as about the backbone: an attribute classifier learns condition, and an anomaly is anomalous *within* a condition, so confidence stays high for target anomalies too. Closing the gap needs a density-based model or an explicit domain-adaptation objective, which is the natural follow-up.

TABLE VIII
DOMAIN-GENERALIZATION AUC (%) ON DCASE 2022 TASK 2 UNDER THE ATTRIBUTE-BASED ADAPTATION. SOURCE IS THE SEEN DOMAIN, TARGET IS UNSEEN.

Backbone	Source	Target	Overall
STgram-MFN	65.08	51.11	58.09
mn01 (fine-tuned)	50.37	49.88	50.12

VI. DISCUSSION

A. Why the Smaller Model Quantizes Worse

The result that surprised us most is the ordering under quantization. Fewer parameters should mean an easier integer model. Here the opposite holds. The mn01 backbone is narrow: six convolutional stages at low width, with a 96-dimensional output. Per-tensor activation scales have little room to represent the dynamic range of its features, and error introduced early compounds through the depth. STgram-MFN has wider activations and a spectrogram input whose scale is already bounded, so its quantization error stays small. The practical lesson is blunt: measure the quantized model. A backbone selection made on floating-point accuracy can pick a model that cannot be deployed.

B. Domain Shift

Two obstacles sit between this study and a clean domain-shift column. The DCASE 2020 evaluation campaign ships its test audio without normal/anomaly labels, so those clips cannot be scored at all. The DCASE 2022 set has labels but hides identities, so it needs the attribute-based adaptation described in Section V-H. That adaptation is honest about its own limits: the target-domain result is at chance and the scoring rule differs from the in-domain one, so it marks a direction of travel rather than a drop-in comparison.

C. Deployment Cost

Latency is not the whole deployment story on a battery-powered node. Crudely, energy per inference scales with power times time, so the 5.75 to 9.87 ms latencies in Table VII correspond to a few millijoules per clip on a phone-class accelerator. At a ten-second stride, that is roughly one inference every ten seconds, which keeps the average draw in the low milliwatts even without aggressive duty cycling. The classifier is not the expensive part; sampling and the acoustic frontend are, which is an argument for doing the frontend in fixed-point hardware and sending only the spectrogram through the network.

D. Threats to Validity

Three caveats deserve mention. First, best-epoch selection uses the test set, inherited from the reference; this inflates absolute numbers, though it applies equally to both backbones. Second, we run a single seed, so small differences should not be over-read. Third, we compare one instance of each design; a different AudioSet student might behave differently under quantization.

VII. CONCLUSION

On a controlled DCASE 2020 comparison, a task-specific spectral-temporal backbone keeps its accuracy under 8-bit post-training quantization, while a nine-times-smaller pre-trained AudioSet embedding does not, even after fine-tuning. The size/accuracy tradeoff is real, but it only becomes meaningful once quantization and deployment are taken into account; on floating-point numbers alone one would choose differently. Three directions follow: revisit the frozen embedding with an identity-agnostic scoring rule, push the ladder down to microcontroller-class arenas, and repeat the comparison under a proper domain-shift formulation.

APPENDIX A PER-EPOCH TRAJECTORIES

Table IX lists the test AUC of the three floating-point runs at every tenth epoch, which is the data behind Figure 4. The frozen run is non-monotone and peaks early; the fine-tuned and reference runs are stable.

TABLE IX
TEST AUC (%) PER EPOCH FOR THE THREE FP32 RUNS.

Epoch	STgram-MFN	mn01 (ft)	mn01 (frozen)
10	89.91	78.10	64.36
20	89.96	79.32	62.72
30	90.21	80.17	53.88
40	90.01	79.19	55.22
50	90.51	79.54	57.70
60	91.05	80.18	58.54
70	90.93	81.36	58.33
80	90.80	82.22	58.60
90	90.91	82.82	58.49
100	91.45	83.16	58.52

APPENDIX B CAPACITY-LADDER CONFIGURATIONS

Table X gives the architecture overrides used for each tier. Only mel width, embedding width, the MobileFaceNet width, and the inverted residual table change; the frontend and training protocol are identical.

TABLE X
ARCHITECTURE OVERRIDES FOR THE CAPACITY LADDER.

Tier	c_dim	embed	mfn width
small	64	64	64
normal	128	128	512
large	128	256	512

APPENDIX C CAPACITY LADDER (RESULTS)

Since STgram-MFN wins on accuracy, we scale it along the mel width, the embedding width, and the MobileFaceNet width to build a small ladder. Table XI reports the variants and Figure 6 places them against the mn01 models. The 0.31 M variant already reaches 90.04 AUC; the 1.16 M reference

TABLE XI
CAPACITY LADDER OF THE WINNING BACKBONE.

Variant	Params	AUC
small	309,247	90.04
normal	1,161,279	91.45
large	2,801,087	91.63

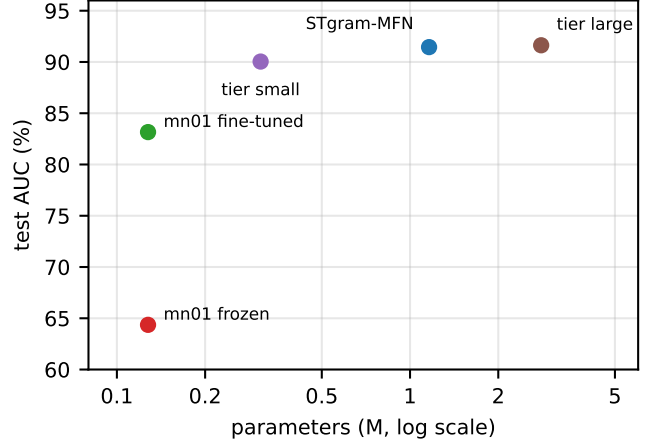


Fig. 6. Parameters against test AUC. The pretrained embedding is small but not accurate; the task-specific ladder trades size for accuracy smoothly.

reaches 91.45 and the 2.80M variant edges it at 91.63. Accuracy rises with capacity and then saturates: tripling the parameters from the small variant buys 1.6 points, and going from 1.16 M to 2.80 M buys 0.18. We therefore stop the ladder at large; a wider variant was attempted but ran out of memory at the shared batch size and, given the flat trend, was not worth re-tuning. The tier checkpoints are released on the model hub as supporting artifacts rather than as claims of the paper.

Algorithm 1 Training, selecting, and quantizing one backbone.

```
1: input: normal clips  $\{(x_i, y_i)\}$ , epochs  $E$ , backbone  $f_\theta$ 
2: for  $e = 1, \dots, E$  do
3:   for each minibatch  $B$  do
4:      $z \leftarrow \text{ArcFace}(f_\theta(x_B), y_B)$ 
5:      $\mathcal{L} \leftarrow \text{CrossEntropy}(z, y_B)$ 
6:      $\theta \leftarrow \theta - \eta_e \nabla_\theta \mathcal{L}$ 
7:   end for
8:   if  $e \bmod 10 = 0$  then
9:     score test clips by Eq. 1; retain best on AUC
10:  end if
11: end for
12: export the best  $f_\theta$  to ONNX
13: calibrate scales on 200 normal clips; apply Eq. 2
14: re-evaluate AUC on the INT8 graph; report  $\Delta\text{AUC}$ 
15: return best FP32 scorer, INT8 scorer, and  $\Delta\text{AUC}$ 
```

REFERENCES

- [1] Y. Koizumi *et al.*, “Description and discussion on DCASE2020 challenge task2: Unsupervised anomalous sound detection for machine condition monitoring,” in *Proc. DCASE Workshop*, 2020. arXiv:2006.05822.
- [2] H. Purohit *et al.*, “MIMII dataset: Sound dataset for malfunctioning industrial machine investigation and inspection,” in *Proc. DCASE Workshop*, 2019. arXiv:1909.09347.
- [3] Y. Liu, J. Guan, Q. Zhu, and W. Wang, “Anomalous sound detection using spectral-temporal information fusion,” in *Proc. IEEE ICASSP*, 2022. arXiv:2201.05510.
- [4] F. Schmid, K. Koutini, and G. Widmer, “Efficient large-scale audio tagging via transformer-to-CNN knowledge distillation,” in *Proc. IEEE ICASSP*, 2023. arXiv:2211.04772.
- [5] A. Howard *et al.*, “Searching for MobileNetV3,” in *Proc. IEEE ICCV*, 2019. arXiv:1905.02244.
- [6] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, “ArcFace: Additive angular margin loss for deep face recognition,” in *Proc. IEEE CVPR*, 2019.
- [7] M. Nagel *et al.*, “A white paper on neural network quantization,” 2021. arXiv:2106.08295.
- [8] G. Hinton, O. Vinyals, and J. Dean, “Distilling the knowledge in a neural network,” 2015. arXiv:1503.02531.
- [9] S. Han, H. Mao, and W. J. Dally, “Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding,” in *Proc. ICLR*, 2016.
- [10] R. Chalapathy and C. K. Chawla, “Deep learning for anomaly detection: A survey,” 2019. arXiv:1901.03407.
- [11] K. Dohi *et al.*, “Description and discussion on DCASE 2022 challenge task 2: Unsupervised anomalous sound detection for machine condition monitoring including domain shifts,” in *Proc. DCASE Workshop*, 2022.
- [12] B. Jacob *et al.*, “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in *Proc. IEEE CVPR*, 2018.
- [13] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted residuals and linear bottlenecks,” in *Proc. IEEE CVPR*, 2018.
- [14] ONNX Runtime Contributors, “ONNX Runtime: Cross-platform, high performance scoring engine for ONNX models,” software, 2024. [Online]. Available: <https://onnxruntime.ai>
- [15] TensorFlow Authors, “TensorFlow Lite for Microcontrollers,” software, 2024. [Online]. Available: <https://www.tensorflow.org/lite/microcontrollers>
- [16] Qualcomm Technologies, “Qualcomm AI Hub: On-device model profiling and deployment,” software, 2024. [Online]. Available: <https://aihub.qualcomm.com>
- [17] Y. Liao and N. Evans, “A survey of anomalous sound detection,” 2023. arXiv:2304.02591.
- [18] A. Goel, S. Berwind, and M. Bornschlegel, “Unsupervised anomalous sound detection using autoencoders and attention,” in *Proc. DCASE Workshop*, 2020.
- [19] T. Fujimura *et al.*, “Anomalous sound detection meets noise-aware self-supervised learning,” 2026. arXiv:2608.00447.
- [20] T. Fujimura *et al.*, “Pseudo-label distillation for discriminative anomalous sound detection,” 2026. arXiv:2607.16678.
- [21] P. Gao *et al.*, “UD-ASD: A unified diffusion model for anomalous sound detection,” 2026. arXiv:2607.12576.
- [22] G. Mkrtchian *et al.*, “Training-free model selection and domain-aware score calibration for first-shot anomalous sound detection,” 2026. arXiv:2607.04526.
- [23] T. Nishida *et al.*, “Description and discussion on DCASE 2026 challenge task 2: Noise-aware unsupervised anomalous sound detection,” in *Proc. DCASE Workshop*, 2026. arXiv:2606.01578.
- [24] Y. Zhang *et al.*, “ECHOv2: A frequency-structured pre-trained acoustic representation model with cross-band modeling,” 2026. arXiv:2607.10596.
- [25] S. Somvanshi *et al.*, “From tiny machine learning to tiny deep learning: A survey,” 2025. arXiv:2506.18927.
- [26] G.-H. Wen *et al.*, “Do time-series foundation models pay off for industrial monitoring? A cost-aware empirical study,” 2026. arXiv:2608.22968.